

Promptanalyse und professioneller Masterprompt

Projekt: PROVOWARE LeerTool - generisches, inhaltlich leeres Tool-Grundgerüst

Referenzdesign: grüne Neon-Variante unten links. Korrektur: ausdrücklich kein Dateifinder und keine Dateisuch-Fachlogik.



Abbildung: visuelle Zielreferenz für Layout, Farbsystem, Kontrast, Rundungen und Panelstruktur.

1. Hochschulische Anforderungsanalyse

Zielkern. Der Auftrag verbindet visuelle Rekonstruktion, ein wiederverwendbares Projektgerüst und einen professionellen Entwicklungs- sowie Betriebsprozess. Für eine belastbare Umsetzung werden diese Ebenen getrennt spezifiziert.

Wesentliche Korrektur. Die Referenz ist keine fachliche Spezifikation. Dateifinder-Inhalte sind nur Stilträger. Das Ergebnis ist ein generisches LeerTool, dessen Arbeitsbereiche absichtlich ohne Fachfunktion bleiben.

Technischer Zielkonflikt. Reines HTML/CSS kann weder Projektordner anlegen noch Logs rotieren, Git-Releases verwalten oder Debugberichte im Standardeditor öffnen. Deshalb bleibt die UI HTML/CSS/Vanilla-JS, ergänzt um eine kleine Python-Standardbibliotheks-Schicht.

Wartbarkeit. Die Basishülle verzichtet bewusst auf Frontend-Framework, Node-Buildprozess und Drittanbieter-Runtime. Das minimiert Abhängigkeiten, Startaufwand und Fehlerquellen.

Automatisierung. Subagenten werden als deterministische Prüfrollen umgesetzt: Preflight, UI, Accessibility, Security, Evidence/Screenshot und Release. Das ist reproduzierbar und offline nutzbar.

Sicherheit. Keine stillen Systemeingriffe: kein sudo, Server nur auf 127.0.0.1, Content Security Policy, keine Telemetrie und Schreibzugriff nur im Projekt-/Registry-Kontext.

2. Präzisierungen und ergänzte Anforderungen

Thema	Präzisierung
Fachinhalt	Keine Dateisuche. Nur neutrale Platzhalter und Layoutflächen.
Projektordner	Erststart fragt Toolname ab; Standard: /home/\$USER/PROVOWARE/Toolname; persistent gespeichert.
Start	Doppelklick-Starter plus start.sh; Python 3.10+ als einzige notwendige lokale Runtime.
Qualität	Gates für Struktur, JSON, Zeilenlimits, A11y-Basis, Security-Basis und Runtime-Dateien.
Iterationen	State-Zähler; bei 10, 20, 30 ... automatisch UI-Screenshot.
Patches	Vorprüfung mit git apply --check; nur kollisionsfrei anwenden; danach validieren und bei Fehler zurückrollen.
Versionierung	SemVer, lokale Projekt-Registry und globale PROVOWARE-Registry.
Diagnose	Rotierende Logs und TXT-Providebugbericht bei Ende, Absturz oder Nichtstart.
Barrierefreiheit	Landmarks, Skip-Link, Tastaturfokus, Focus Ring, Reduced Motion, responsive Breakpoints.

3. Professionell verbesserter Masterprompt

Rolle und Ziel

Erstelle ein generisches PROVOWARE LeerTool als wiederverwendbares Projektgrundgerüst. Die grüne Referenzvariante unten links ist ausschließlich die visuelle Vorlage für Layout, Design, Erscheinungsbild, Farben, Abstände, Rundungen, Panelstruktur und Elementformen. Übernimm keinerlei Dateifinder-Fachlogik, Suchfunktion oder dateibezogene Inhalte.

UI-Scope

Baue eine leere, modulare Desktop-artige Oberfläche mit linker Navigation, Kopfbereich, zentraler Werkzeugzone, großem Hauptpanel, rechtem Neben-/Vorschau-panel und unterer Status-/Aktionsleiste. Sichtbare Inhalte bleiben neutral und dienen nur als Platzhalter. Nutze das grüne Neon-Farbsystem der Referenz, dunkle Flächen, feine grüne Konturen, leichte Glows, klare Kontraste und kompakte Rundungen. Das Layout muss responsive bleiben.

Technische Basis

Verwende semantisches HTML, wartbares CSS und nur wenig Vanilla-JavaScript. Ergänze für lokale Betriebsfunktionen ausschließlich eine kleine Python-Standardbibliotheks-Schicht. Vermeide einen Node-Buildprozess und unnötige Drittanbieter-Abhängigkeiten. Halte Dateien klein und erzwingende definierte Zeilenlimits.

Erststart und Projektpfad

Implementiere eine laiengerechte Klick-und-Start-Routine. Beim ersten Start wird ein Toolname abgefragt. Standardziel ist /home/\$USER/PROVOWARE/Toolname. Der gewählte Pfad wird persistent gespeichert. Existiert der Zielordner nicht, wird er erstellt; existiert er, wird seine Struktur geprüft. Schreibrechte und notwendige Runtime-Bedingungen werden validiert. Es werden keine administrativen Rechte und kein sudo verwendet.

Entwicklungsworkflow und Gates

Definiere deterministische Prüfrollen für Preflight, UI-Struktur, Accessibility, Security, Runtime, Screenshot-Evidence und Release. Jede Iteration folgt: planen -> Kollisionen prüfen -> Änderung/Patch vorbereiten -> Vorvalidierung -> anwenden -> Nachvalidierung -> Iteration abschließen. Fehler blockieren den nächsten Release-Schritt.

Patch-System

Patches werden vor der Anwendung vollständig ermittelt und als Plan erfasst. Git-Patches werden zuerst mit git apply --check auf Kollisionen geprüft. Nur wenn alle Patches kollisionsfrei sind, werden sie angewendet. Anschließend laufen alle Qualitäts-Gates. Bei fehlerhafter Validierung wird auf den sicheren Checkpoint zurückgesetzt.

TODO-Management

Führe zwei getrennte Listen: geplante TODOs und zusätzlich erkannte TODOs. Neu erkannte Punkte erhalten stabile IDs, werden auf Abhängigkeiten und Dateikollisionen geprüft und anschließend konfliktfrei in kommende Iterationen eingeplant.

Versionierung und Registry

Nutze Semantic Versioning. Pflege eine projektlokale Registry mit Version, Designprofil und Iterationsstand sowie eine lokale globale PROVOWARE-Registry. Initialisiere Git lokal, wenn Git verfügbar ist. Keine automatische Remote-Veröffentlichung.

Screenshot-Regel

Nach jeder 10. abgeschlossenen Iteration wird automatisch ein Screenshot der UI in 1536 x 1024 erzeugt und im Projekt dokumentiert. Ein manueller Screenshot-Befehl muss ebenfalls vorhanden sein.

Logging, Fehlerhandling und Self-Repair

Nutze rotierende Logs mit eindeutigen Fehlercodes. Bei normalem Ende, Absturz oder Nichtstart wird sofort ein TXT-Profidebugbericht erzeugt, der Umgebung, Fehlercode, letzte Logzeilen, Ursache und konkrete Lösungsvorschläge enthält und möglichst im Standardeditor geöffnet wird. Sichere Self-Repair-Schritte dürfen angeboten werden; destruktive oder privilegierte Reparaturen erfolgen nie still.

Barrierefreiheit, Tastatur und Sicherheit

Verwende semantische Landmarken, Skip-Link, sichtbare Fokuszustände, sinnvolle ARIA-Beschriftungen, Reduced-Motion-Unterstützung und Tastaturzugriff. Binde den lokalen Webserver ausschließlich an 127.0.0.1, setze eine Content Security Policy und vermeide Telemetrie oder externe Netzwerkzugriffe.

Abnahmekriterien

Das Projekt startet per Doppelklick oder start.sh, bleibt fachlich leer, zeigt das grüne Referenzdesign, läuft ohne

Frontend-Framework, besteht die Qualitäts-Gates, besitzt getrennte TODOs, SemVer/Registry, Patch-Preflight, Iterationszähler, Screenshot-Automation, persistente Projektpfadwahl und einen automatisch öffnenden Debugbericht.

4. Sinnvolle Ergänzungen zum Ausgangsprompt

- **Rollback-Sicherheit:** Änderungen dürfen bei fehlgeschlagener Validierung nicht halb angewendet bleiben.
- **Offline-First:** Die Basishülle benötigt keine Cloud, Telemetrie oder externe APIs.
- **Security by Default:** lokale Bindung, CSP, kein eval, keine stillen Systeminstallationen.
- **Zustandsmigration:** Konfigurationsdateien tragen Schema-Versionen.
- **Trennung von Runtime und Dev-Tools:** Normaler Start bleibt unabhängig von Screenshot- und Release-Werkzeugen.
- **Nachvollziehbarkeit:** Automatisierte Schritte schreiben Logs und aktualisieren den Projektzustand.
- **Erweiterung ohne Umbau:** Fachmodule werden später ergänzt, ohne die Basisschale neu strukturieren zu müssen.

5. Anwendungsplan

1. Visuelle Referenz auf die untere linke, grüne Variante begrenzen.
2. Generische Leerschaale mit denselben Hauptproportionen und dem grünen Designsystem umsetzen.
3. Startup, Projektpfad, Registry, Logging und Debugbericht ergänzen.
4. Qualitäts-Gates, TODO-Trennung, Patch-Preflight, Iterations- und Screenshot-Mechanik ergänzen.
5. Projekt validieren, lokal rendern und als portable ZIP-Datei ausgeben.